

# Lecture Notes on Deep Neural Networks

Jawwad Ahmed Shamsi  
FAST-NUCES, Pakistan

March 30, 2026

## 1 Introduction

As discussed earlier, a sequence to sequence network, is a type of network, which takes a series of input and generates a series-based output. It essentially consist of an encoder and a decoder.

An encoder is a type of neural network, which takes an input and maps it into an encoding scheme. A decoder takes the encoded scheme as an input and decodes it to determine the desired output.

### 1.1 Examples of Seq 2 seq networks

There are many applications (and types) of seq 2 seq networks

In general, a seq 2 seq network follows an encoder decoder architecture.

Let's study the image captioning system in a little more detail .

It takes a fixed size vector as input and generates a sequence of any length as output.

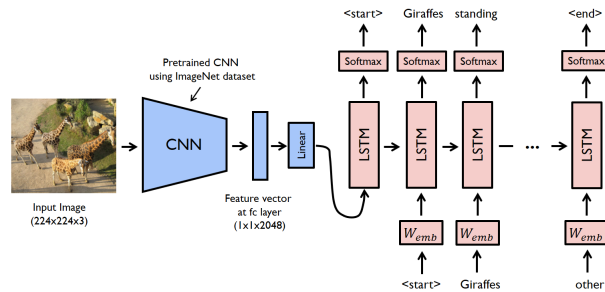


Figure 1: Image Captioning  
[imagecaptioning]

Another example is sequence to vector model which takes a variable input and generates a fixed size output. A sentiment analysis system is an example of such a system [sequencetosequence].

A machine translation system is an example of variable size input and variable size output.

An encoder in all the above mentioned examples is a system, which encodes the input in a given format, whereas a decoder generates the output in the desired format.

### 1.2 RNN vs LSTM

1. Slow in training
2. long sequences vanish or explode

| Type         | Example             |
|--------------|---------------------|
| one to many  | Image captioning    |
| many to one  | Sentiment analysis  |
| many to many | Machine Translation |

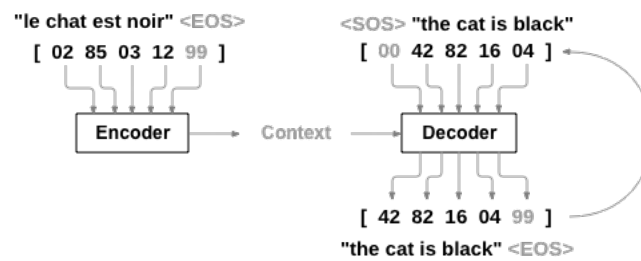


Figure 2: Machine Translation  
[sequencetosequence]

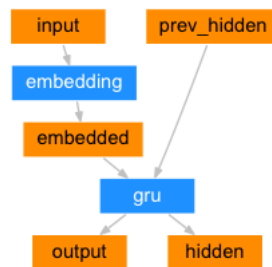


Figure 3: Encoder Network  
[sequencetosequence]

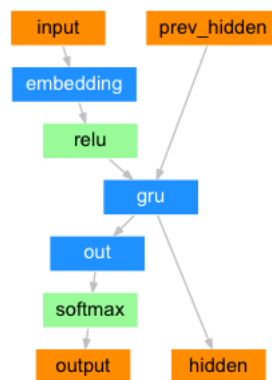


Figure 4: Decoder Network  
[sequencetosequence]

LSTM solves the second problem

However, the first problem persists. LSTMs are slow and are difficult to train

LSTM are sequential. therefore they are slow.

## 2 Transformers

A transformer is a non-recurrence model. That is, unlike RNNs/LSTM/GRU which relies on recurrence, a transformer is not based on recurrence. Since transformers do not rely on recurrence, all tokens in a sequence can be processed in parallel. This significantly improves training efficiency compared to RNNs and LSTMs. A transformer architecture could be well understood from figure 5. It consists of an Encoder unit and a Decoder Unit. A transformer consists of six such units.

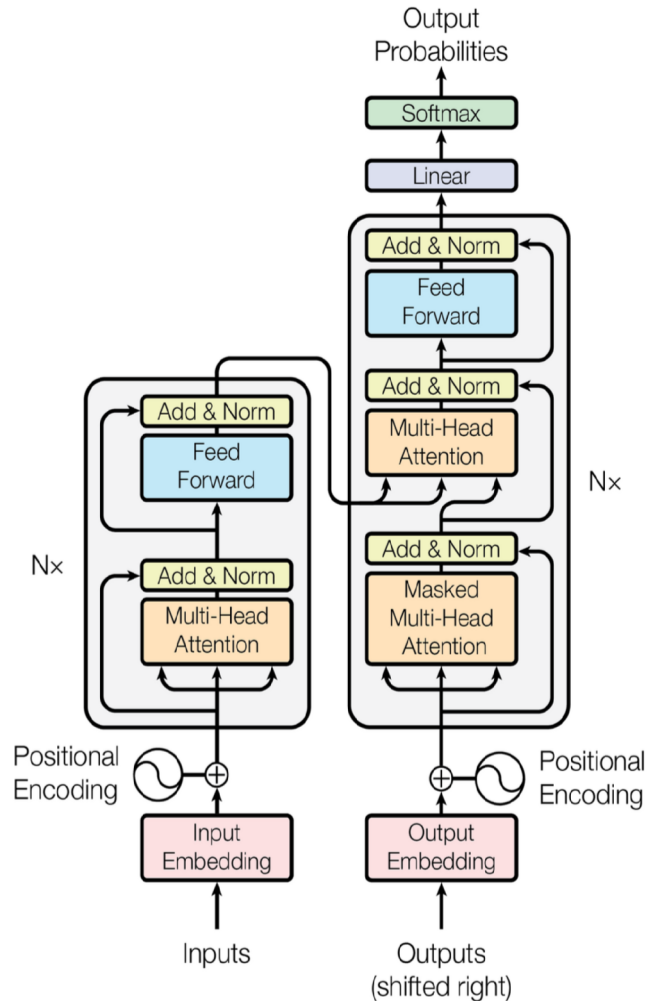


Figure 5: Transformers Architecture  
[transformers]

Figure 5 illustrates the basic architecture of transformers. The encoder consists of two blocks, i.e., a multi-head attention and a feed forward block. The decoder contains an additional block.

Let's study these blocks (units) in detail. Let's begin with an example of English to Urdu Translation.

### 2.1 Encoder

As per the research paper, there are six encoder units stacked together. Each unit has an Attention block and a Feed Forward block. There is an additional process of converting word embeddings into positional encoding.

Since a transformer is a non-recurrent neural network, it cannot capture context related to positions. This information has to be captured through positional encoding.

1. **Positional Encoding:** The main purpose of positional encoding is to add context of positions in a sentence. This unit takes word embeddings as input and outputs positional embeddings. The process of positional encoding is explained in section 2.5
2. **Multi-head Attention:** This unit is responsible for generating attention vectors (weights). Multi-head implies that multiple head weight vectors are generated. The basic attention mechanism of a transformer is based on scaled dot-product attention. Details are mentioned in section 2.7.
3. **Feed Forward:** A feed forward unit is connected to the attention block. It is used to transform the attention vector generated by the attention block into a form which is acceptable to the next encoder block or decoder block.

## 2.2 Decoder block

The purpose of the decoder is to predict the next word. During the training phase, we feed the output urdu sentence to the decoder. This is done by computing input embedding and adding the positional vector. Following are the main blocks on the decoder side:

1. **Masked multi-head attention:** This is the attention block on the decoder side. It is based on scaled dot-product. It generates attention vector for every word in the output language, i.e, Urdu. This attention vector will represent that how much each word in the output sentence is related to every word in the same sentence. However, it also involves masking. Masking means that decoder will mask the embedding for words, which have not been generated by the decoder.
2. **Encoder Decoder Attention Block:** There is another attention block. It takes input from the encoder and input from the above attention block. That is, it takes input from the encoder as well as the decoder side and determines attention vector from input to output. This block computes attention between the encoder outputs (source sequence) and the decoder inputs (target sequence), enabling the decoder to focus on relevant parts of the input sentence.  
The output of this block is attention vectors for every English and Urdu sentences. These vector provides relationships with respect to both the languages.
3. **Feed Forward Block:** The purpose of this block is to make the output vector more digestible either to the next decoder block or to the linear layer.
4. **Linear Layer:** The linear layer used to expand dimensions for the output language.
5. **Softmax Layer:** This layer transforms the output into probability distributions. The word with the highest probability is selected.

The encoder decoder model is executed over several time steps until we reach the end of sentence.

## 2.3 Training

For training, we use sentences in both the languages. These sentences are translations of each other. For the English to Urdu Translation, we input the English sentence on the Encoder side, whereas on the decoder side, we input the Urdu sentence. However, the decoder input will be masked and shifted to the right by one position

The main reason for shifting is to ensure that our model does not learn to copy the decoder input.

If the sequence is not shifted right then the model will learn copying the decoder input. This is because for position  $j$  (character/word) in the encoder sequence, the model will learn to copy the decoder input for position  $j$ .

The shifting of the decoder input will ensure that we will train our model such that given the encoder sequence and the decoder sequence from 1, ...,  $j-1$ , the model can predict the  $j$ th sequence.

This process ensure that our model does not learn copy pasting of the translated sequence, instead it learns to predict the next word in the sequence

During training the first word of the decoded sentence is filled with a 'start of sentence' token. The end of sentence token is also attached to mark the end of the sentence.

For loss function, we will consider the target sentence (i.e., the Urdu sentence) without shifting and with the end of token sentence.

Further, if during the training process, a decoder estimates incorrectly, even then the corrected word is given in the input and the incorrect word is not considered. This phenomenon is called **teacher forcing**. The rationale is that similar to a teacher, which tries to teach a student by giving hints. During inference, teacher forcing is not used; instead, the model relies on its own previous predictions, which may introduce error accumulation. The whole process of training can be understood from figure 6

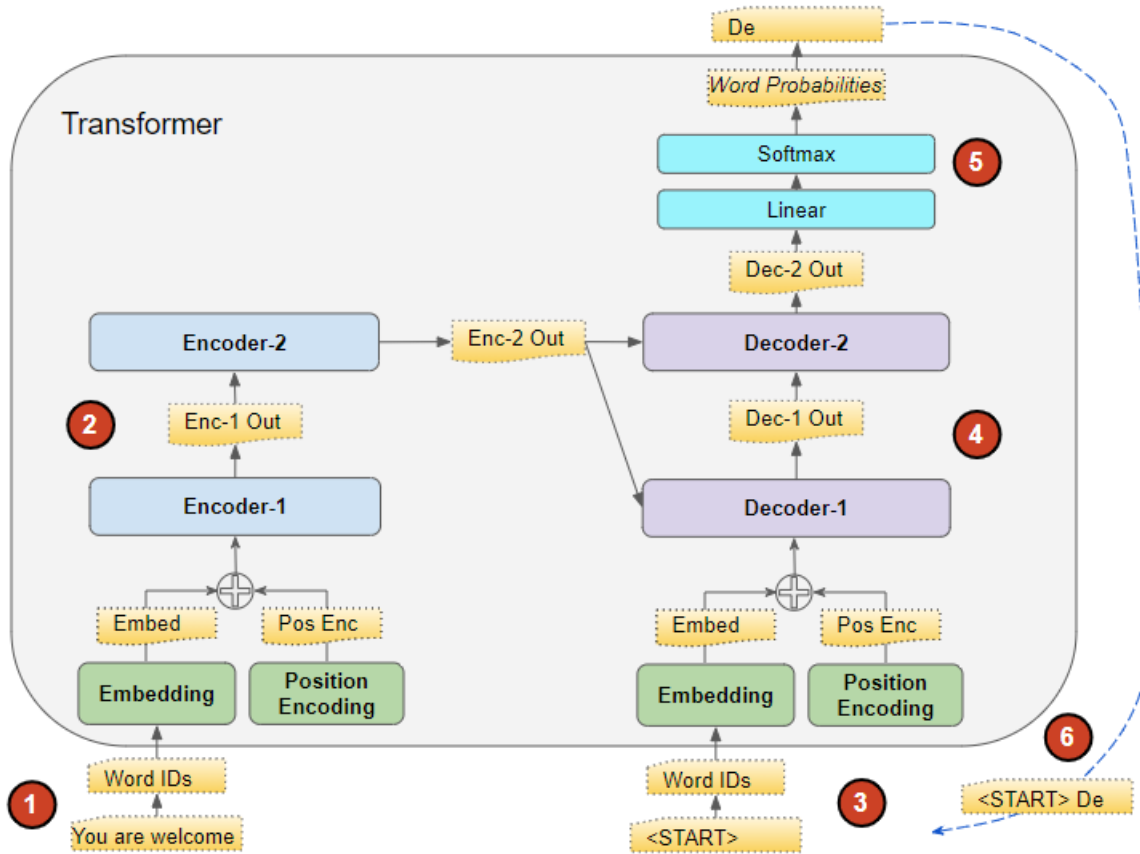


Figure 6: Transformer Training  
[transformers-details1]

1. The sequence of words at the input layer is converted into Embeddings (such as word2vec). This sequence is then transformed with Positional Encoding and fed to the Encoder. Since the transformer is not a recurrent network, it cannot learn positions.
2. At the encoding site, there is a stack of encoders ( $N=6$  in the original paper), which produces an encoded representation.
3. At the Decoder site, the words which have been generated or predicted so far are sent to the decoder as a sequence. This is converted into embeddings with positional encoding. For the very first word, a start of sentence tag is sent. In this way, the decoder input is considered as shifted right.
4. A stack of decoders process this. The stack takes input from the encoded layer output (encoded representation). This produces the encoded representation of the output or the target sequence.
5. The decoder outputs probabilities. The total number of neurons at the output layer is the number of unique words in the corpus. In this manner, a sequence is generated at the output sequence.
6. AT the output, loss function is used to compare the predicted words with the actual output. This is used during back-propagation.

## 2.4 Inference or Testing

Inference or testing the transformer the machine translation problem is trivial. On the decoder side, we begin with the start of the sentence token and keep adding the next word with the highest probability [maxinetransformer].

The testing (or inference) part is similar except that instead of sending the actual sequence, we share the predicted (So far) sequence.

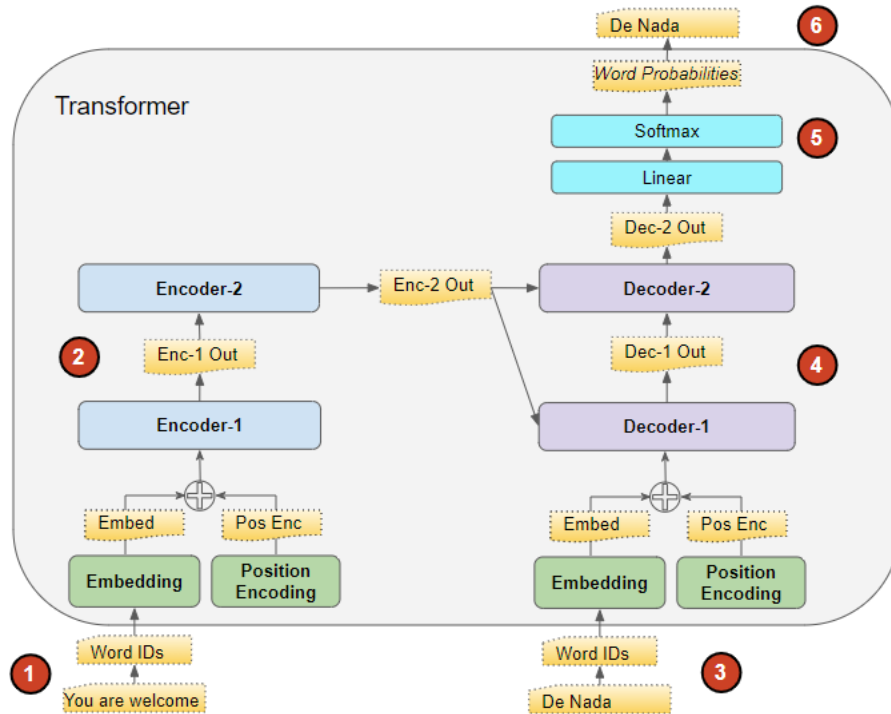


Figure 7: Transformer Inference  
[transformers-details1]

Detail steps are mentioned below:

1. Input the English sentence (encoder output) with the start of the sentence in the decoder. It will output the first word in the decoder sequence (Urdu).
2. Add the first word (output above) in the sequence of decoder
3. Now input the encoded sentence with the above mentioned decoder sequence (start of the sentence and the first word) to predict the next word in the sequence.
4. Iterate by appending the decoder sequence and predicting the next word in the decoder sequence by considering the complete encoder sequence and the current state of the decoder sequence.

## 2.5 Positional Encoding

Positional encoding is used to inject information about the order of words in a sequence. Unlike RNNs, transformers process all tokens in parallel and therefore do not inherently capture positional relationships.

To address this, we add a positional encoding vector to each word embedding so that the model can distinguish between different positions in a sequence.

For instance,

AJ's dog is a cutie [position 2] AJ looks like a dog [position 5]

The module captures the positional information of words in a sentence. Each position has a unique value in the sentence. For long sequences, indices can grow, therefore a vector is used instead of an index.

This layer outputs a matrix, which is used for positional encoding. Each row of the matrix represents positional information of the sequence.

$L$ : length of the sequence

The variable  $k$  denotes the position of the object in the input sequence

$d$ : It is the dimension of the output embedding space

$P(k, j)$ : This denotes the position function, which is used to compute positional embedding.  $(k, j)$  denotes input sequence to index  $(k, j)$  of the positional matrix

$n$ : It is a user defined scalar. The original paper uses the value of 10,000 .

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right) \quad (1)$$

$$P(k, 2i + 1) = \cos\left(\frac{k}{n^{2i/d}}\right) \quad (2)$$

$i$ : It is use to map column indices

$$0 \leq k < L$$

where:

- $k$  is the position of the token in the sequence,  $0 \leq k < L$
- $i$  is the dimension index,  $0 \leq i < \frac{d}{2}$
- $d$  is the embedding dimension
- $P(k, j)$  denotes the positional encoding value at position  $k$  and dimension  $j$

Both sine and cosine functions are used for mapping.

The use of sine and cosine functions allows the model to capture relative positions. These functions produce periodic patterns at different frequencies, enabling the model to learn relationships such as distance between words.

Additionally, using continuous functions allows the model to generalize to sequence lengths longer than those seen during training.

Even indices use sine functions while odd indices use cosine functions. This alternating pattern allows each position to be uniquely represented and helps the model distinguish between different dimensions.

**Example** Let us consider an example to illustrate positional encoding.

Sentence: "I am a robot"

Embedding dimension:  $d = 4$ , scaling factor:  $n = 100$

Each word is assigned a position index, and the positional encoding vector is computed for each position using the above equations.

The variable  $i$  is used to index the dimensions.

All values are computed in radians.

From the above matrix, we observe that each position is represented by a unique combination of sine and cosine values across dimensions.

Even small changes in position result in different encoding vectors, allowing the model to distinguish between positions and learn relative relationships.

The positional encoding vector is added to the word embedding vector: Each position is mapped to a unique vector representation that encodes both absolute and relative positional information.

$$\text{Final Input} = \text{Embedding} + \text{Positional Encoding}$$

The addition allows positional information to be incorporated without increasing the dimensionality of the embeddings. Thus, positional encoding enables transformers to incorporate sequence order information while maintaining parallel computation.

## 2.6 Overall Process of Transformer

The overall workflow of a transformer model during training and inference can be summarized as follows:

1. The input sentence is converted into embeddings.
2. Positional encoding is added to incorporate order information.
3. The embeddings are passed through the encoder stack.

Positional Encoding  
Matrix with  $d=4$ ,  $n=100$

| Sequence | Index of token, $k$ | $i=0$                        | $i=0$                         | $i=1$                         | $i=1$                         |
|----------|---------------------|------------------------------|-------------------------------|-------------------------------|-------------------------------|
| I        | 0                   | $P_{00}=\sin(0)$<br>= 0      | $P_{01}=\cos(0)$<br>= 1       | $P_{02}=\sin(0)$<br>= 0       | $P_{03}=\cos(0)$<br>= 1       |
| am       | 1                   | $P_{10}=\sin(1/1)$<br>= 0.84 | $P_{11}=\cos(1/1)$<br>= 0.54  | $P_{12}=\sin(1/10)$<br>= 0.10 | $P_{13}=\cos(1/10)$<br>= 1.0  |
| a        | 2                   | $P_{20}=\sin(2/1)$<br>= 0.91 | $P_{21}=\cos(2/1)$<br>= -0.42 | $P_{22}=\sin(2/10)$<br>= 0.20 | $P_{23}=\cos(2/10)$<br>= 0.98 |
| Robot    | 3                   | $P_{30}=\sin(3/1)$<br>= 0.14 | $P_{31}=\cos(3/1)$<br>= -0.99 | $P_{32}=\sin(3/10)$<br>= 0.30 | $P_{33}=\cos(3/10)$<br>= 0.96 |

Positional Encoding Matrix for the sequence 'I am a robot'

Figure 8: Encoder Network  
[posencoding]

4. Each encoder layer applies self-attention followed by a feed-forward network.
5. The encoder produces contextualized representations of all input tokens.
6. On the decoder side, previously generated tokens are used as input.
7. Masked self-attention ensures that future tokens are not visible.
8. Encoder-decoder attention allows the decoder to focus on relevant input tokens.
9. The output is passed through a linear layer and softmax to generate probabilities.
10. The next token is selected based on the highest probability.

## 2.7 Scaled-dot Attention

Transformer uses self attention mechanism. In such a mechanism, the input sequence learns attention weights (attention) for its own input sequence (self). In other words, it is an attention mechanism, which relates different positions of a single sequence and determines attention representation of the same sequence.

The self-attention mechanism of transformers are based on scaled-dot attention.

We will now learn the scaled-dot attention mechanism of transformers. The mechanism is based on query  $Q$ , Keys  $k$ , and Values  $V$ . This is similar to the concept of a web search. When we query a web search engine, it will map the query to a set of keys. Each key can consist of title of the page, description etc. whereas the search engine will select web pages with best results or best values. The same notion is applied to the attention mechanism of transformers. Figure 9 shows this concept. In transformers attention (weights) are used to match keys against values.

In the original transformers paper, keys are used to define the attention weights to look at the data and the values are used as the information that will actually be obtained [adaloglou2021transformer].

Figure 11 illustrates the scaled dot attention mechanism of transformers.

Let us consider an example. "Hello I like you". A trained self-attention layer will associate the word "like" with the words "I" and "you" with a higher weight than the word "Hello". Linguistically, the three words share a subject-verb-object relationship as well [adaloglou2021transformer].

Attention is defined as a mapping from a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors.

Let us assume that  $q$  and  $k$  denotes vectors of dimension  $d_k$ , containing the queries and keys, respectively, and  $v$  denoting a vector of dimension  $d_v$  containing the values. Therefore,  $Q$ ,  $K$ , and  $V$  denotes matrices of queries, keys and values, respectively.

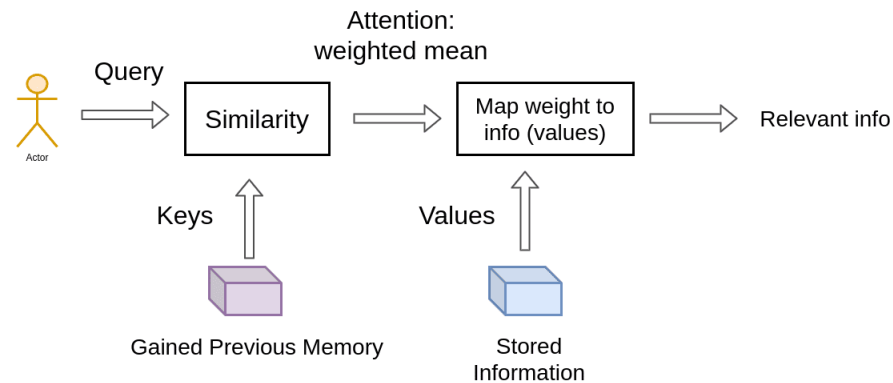


Figure 9: Query vs Key vs Value  
[adaloglou2021transformer]

Self-attention  
Probability score matrix

|       | Hello | I   | Like | you  |
|-------|-------|-----|------|------|
| Hello | 0.8   | 0.1 | 0.05 | 0.05 |
| I     | 0.1   | 0.6 | 0.2  | 0.1  |
| Like  | 0.05  | 0.2 | 0.65 | 0.1  |
| you   | 0.2   | 0.1 | 0.1  | 0.6  |

Softmax(Attention)  
equation

Figure 10: Self Attention Example

### Scaled Dot-Product Attention

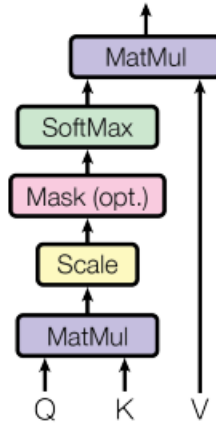


Figure 11: Scaled dot attention  
[vaswani2017attention]

We can describe scaled dot attention as follows

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (3)$$

We initialize  $q$ , and  $k$  as independent random variables with mean =0 and variance =1 , then their dot product,

$$q \cdot k = \sum_{i=1}^{d_k} u_i v_i \quad (4)$$

has mean 0 and variance  $d_k$

Since we would prefer these values to have variance 1, we divide by  $\sqrt{d_k}$ . If the components of  $q$  and  $k$  are independent with mean 0 and variance 1, then their dot product has variance  $d_k$ . This can lead to large values of  $QK^T$ , pushing the softmax function into regions with very small gradients. To mitigate this, we scale by  $\frac{1}{\sqrt{d_k}}$ .

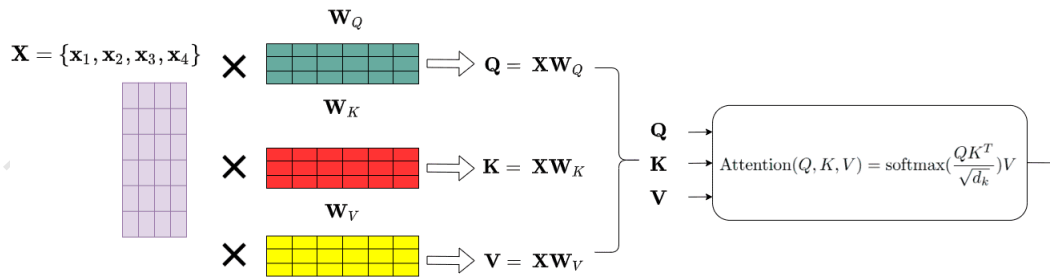


Figure 12: Key-query-value  
[adaloglou2021transformer]

In scaled dot-product attention, the dot products are scaled by  $\frac{1}{\sqrt{d_k}}$  to control their magnitude.

In general, attention mappings are denoted as function of query and a set of key-value pairs. Transformers use a "Scaled Dot-Product Attention" to obtain the context vector.

**Explanation of Figure 13** Figure 11 illustrates the detailed computation of scaled dot-product attention within a transformer layer.

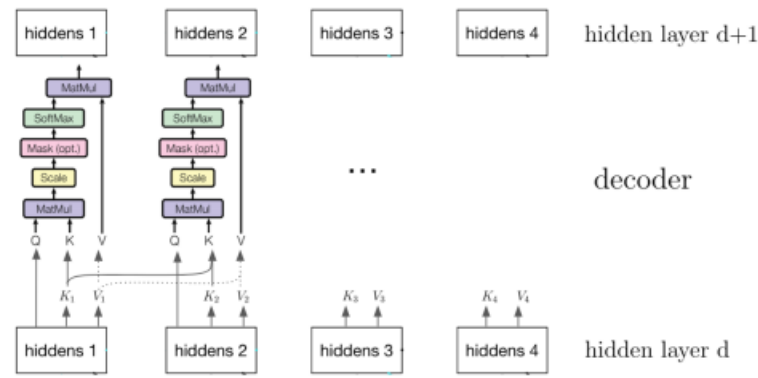


Figure 13: Scaled dot attention Explained details  
[DNN'toronto]

Each input token is projected into Query (Q), Key (K), and Value (V) vectors. The query vector is compared with all key vectors to compute similarity scores, which are then scaled and normalized using the softmax function. These normalized weights are used to compute a weighted sum of the value vectors, producing a context-aware representation.

An important aspect highlighted in this figure is that this attention mechanism is not applied only once. Instead, it is repeated across multiple layers of the transformer (typically  $N = 6$  encoder layers). Each layer takes the output of the previous layer as input and refines the representation further.

Thus, the model progressively builds richer contextual representations, where lower layers may capture local relationships and higher layers capture more abstract and global dependencies.

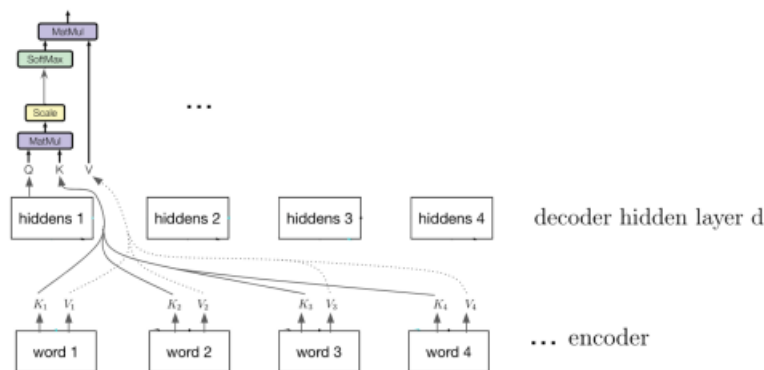


Figure 14: Scaled dot attention Explained  
[DNN'toronto]

**Explanation of Figure 14** Figure 14 provides a higher-level view of how self-attention operates across a sequence of tokens.

Each word in the input sequence is transformed into a vector representation, and self-attention allows every word to interact with every other word in the sequence. This enables the model to capture contextual relationships such as syntactic structure, semantic meaning, and long-range dependencies.

Importantly, this process is not limited to a single layer. In the transformer architecture, multiple encoder (and decoder) layers are stacked on top of each other. Each layer applies self-attention followed by a feed-forward network, refining the representations at every stage.

As a result, the output representations become increasingly context-aware as they propagate through successive layers, allowing the model to learn complex hierarchical relationships in the data.

Figure 13 and 14 illustrates the concept of self-attention in transformers. The attention mechanism shown in Figures 13 and 14 operates within a single layer; in practice, multiple such layers are stacked to form the encoder

and decoder of the transformer architecture.

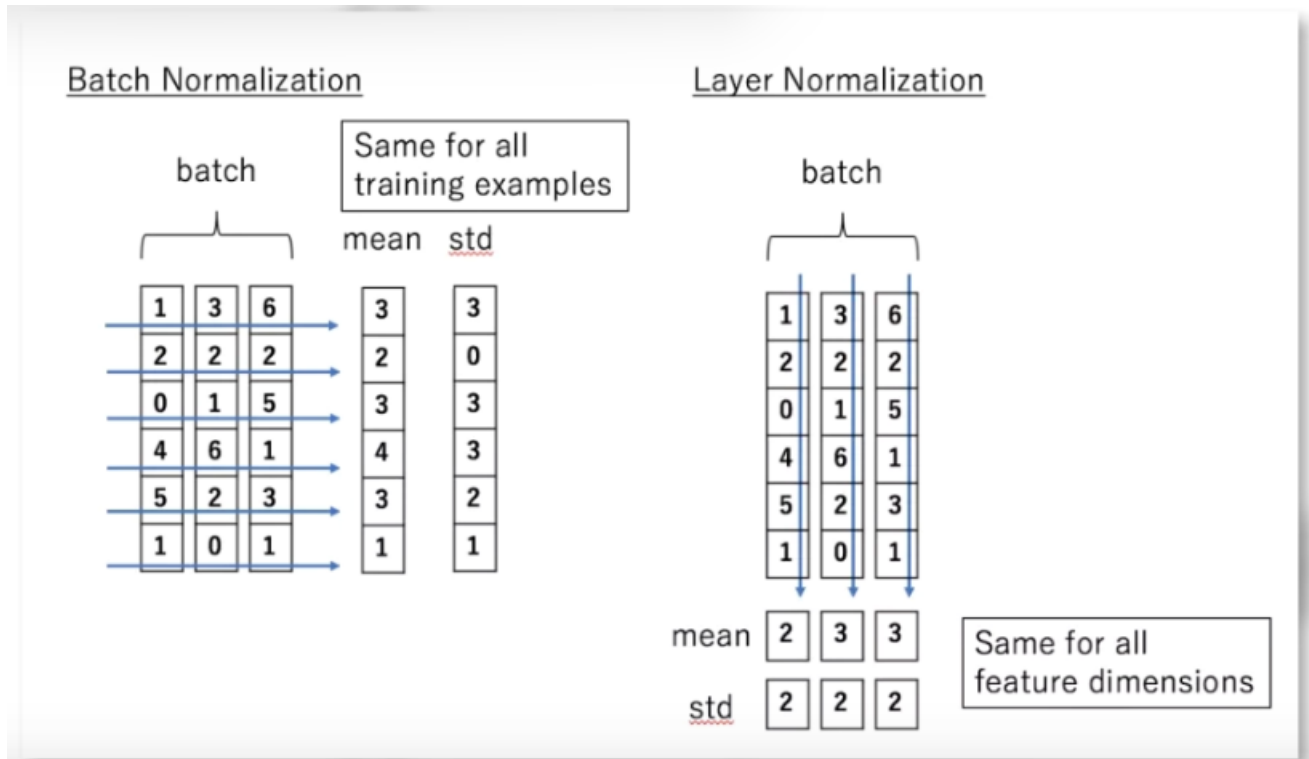


Figure 15: Layer Normalization  
[layer-normalization]

Figure 15 illustrates the concept of layer normalization.

## 2.8 Single-head Attention

Q,K,V are abstract vectors. They extract different components of an input word.

We have Q,K,V vectors for every single word.

we use these to compute attention vectors for every word using

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Figure 16: Scaled dot attention Explained details  
[DNN'toronto]

The transformer mechanism utilizes attention at three instances [transformers'in'NLP]:

1. : Encoder-decoder attention layers. The values are taken from the previous decoder layer, whereas the queries and keys from the encoder output. This attention instance learns weights for each position in the decoder with respect to each position in the input sequence.
2. The self-attention layer contained in the encoder generates random attention matrix values for query, key, and value. Further iterations allow learning of these attention weights. Subsequent layers of encoders takes input from the previous encoder layers.
3. : Self-attention in the decoder is similar. All the values of queries, keys, and values are taken from the previous layer. The attention weights are masked up to the values for the position decoded so far.

|                       |                                                                       |                                           |
|-----------------------|-----------------------------------------------------------------------|-------------------------------------------|
| Feature               | <b>Simple Attention</b>                                               | <b>Self-Attention</b>                     |
| <b>Definition</b>     | Focuses on relevant parts of the <i>encoder</i> input while decoding. | Attention mechanism where a sequence      |
| <b>Used In</b>        | Seq2Seq models with attention (e.g., Bahdanau, Luong).                | Transformers (Encoder & Decoder).         |
| <b>Input Scope</b>    | Decoder attends to encoder outputs.                                   | Each token attends to all tokens in the s |
| <b>Purpose</b>        | Helps decoder align with relevant encoder states.                     | Builds context-aware representations with |
| <b>Weight Sharing</b> | Single attention weight per token.                                    | One set of weights across input sequence  |
| <b>Output</b>         | Weighted sum of encoder hidden states.                                | Weighted sum of input embeddings.         |

## 2.9 Limitation of Single-Head Attention

In single-head attention, each token attends to other tokens using a single set of attention weights. This means that the model learns only one type of relationship at a time.

However, natural language contains multiple types of relationships simultaneously, such as:

- syntactic relationships (subject–verb)
- semantic relationships (meaning)
- positional relationships (nearby vs distant words)

A single attention mechanism may not be sufficient to capture all these relationships effectively.

## 2.10 Why Multi-Head Attention?

Multi-head attention addresses the limitation of single-head attention by allowing the model to learn multiple relationships in parallel.

Instead of computing a single attention distribution, the model computes multiple attention distributions (called heads), each focusing on different aspects of the input sequence.

Each head learns a different representation of the data. For example:

- One head may focus on grammatical structure
- Another head may capture long-range dependencies
- Another head may focus on local context

These multiple representations are then combined to form a richer and more informative representation.

Thus, multi-head attention allows the model to jointly attend to information from different representation subspaces.

Each head operates on a different learned projection of the input, allowing the model to attend to information from different representation subspaces.

## 2.11 Multi-Head Attention

In multi-head attention, instead of applying a single attention function, we project the queries, keys, and values multiple times using different learned linear projections.

For the  $i^{th}$  head:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Each head operates independently and captures different relationships in the input.

The outputs of all heads are concatenated and projected again:

**Key Idea:** Each head attends to different parts of the sequence and learns different contextual relationships, resulting in a richer representation.

## 2.12 Types of Attention: Intuition with Examples

### 1. Simple Attention (Encoder-Decoder Attention)

The decoder attends to encoder outputs.

Example: In translation of “I love you”, when generating the word “love”, the decoder focuses more on the corresponding input word “love”.

## 2. Self-Attention

Each word attends to all other words in the same sentence.

Example: In the sentence “The animal didn’t cross the street because it was tired”, the word “it” attends strongly to “animal”.

## 3. Multi-Head Attention

Multiple self-attention mechanisms run in parallel.

Example: In the same sentence:

- One head may link “it” to “animal”
- Another head may capture “cross” → “street”
- Another head may focus on word order

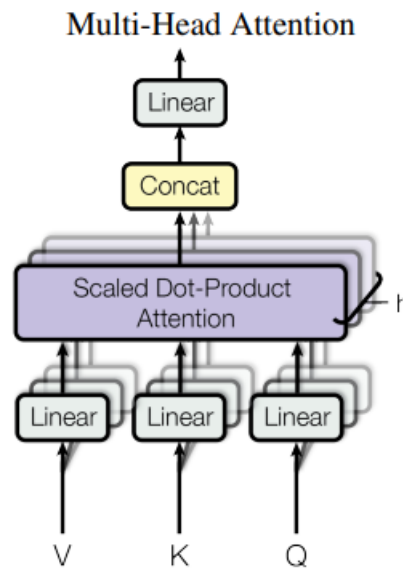


Figure 17: Multi Head Attention  
[vaswani2017attention]

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Figure 18: Concatenation in Multi-head  
[anushaattention]

Figure 20 illustrates the concept of computing multi-head attention.

All attention heads are computed in parallel, allowing the model to capture multiple relationships simultaneously without increasing sequential computation.

For multi-head attention, we have multiple weight matrices, i.e., one for each head.

Since the Feed forward Neural Network is only expecting one attention vector per word, we use another weighted matrix  $W^Z$  to compute final attention vector per word.

Multi-head attention ensures that different parts of the sequence are attended differently. It provides improved learning for positional information as each head will attend different segments of the input and will learn different contextual information.

Each attention head can be implemented in parallel

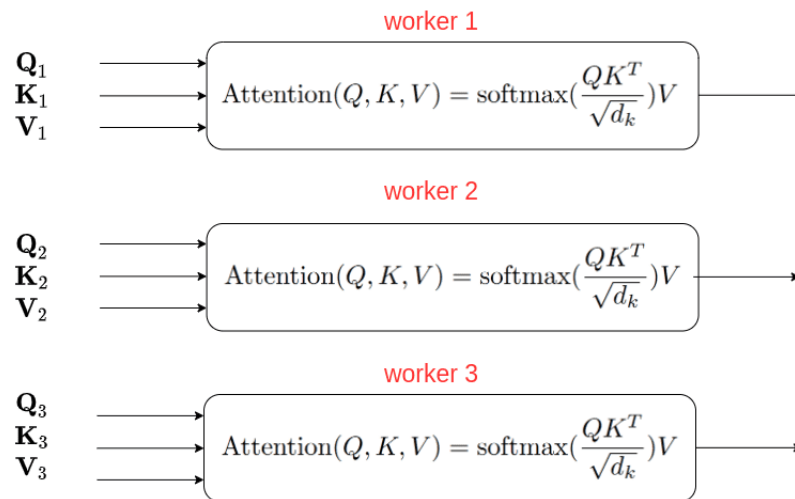


Figure 19: Parallelism in Multi-head attention model  
[adaloglou2021transformer]

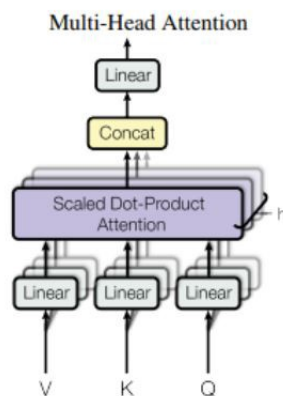


Figure 20: Multi-head-Attention Model  
[anushaattention]

## 2.13 Residual Connection

Transformers also has residual connections. Residual connection serves two main purposes:

1. Knowledge Preservation During forward propagation the input gets modified considerably by the time it reaches the last layer. This may result in loss of information as compared to the input layer in the system. Residual connection will ensure that old information will not get lost. We can add two pieces of information to get the final output.
2. Vanishing Gradient Problem

In summary, Residual connections help in stabilizing training by allowing gradients to flow directly through the network, thereby mitigating the vanishing gradient problem.

They also enable the model to retain important information from earlier layers by adding the input of a layer to its output.

After every layer, we apply some form of normalization. this smoothens out loss surface making it easier to optimize. We prefer layer normalization.

By leveraging parallelism, we can pass all the words together in our encoder block and it will output all the words together.

**Further reading** [transformers-detail].